

AutoJudge: Predicting Programming Problem Difficulty Using Machine Learning

• Introduction

Online competitive programming communities such as Codeforces, CodeChef, and Kattis organize programming problems according to their level of difficulty (Easy, Medium, Hard) and even use an integer-based level of difficulty. The assignment of the level of difficulty is done manually by human expert intervention. This is time-consuming.

This project aims to develop an automated machine learning system that predicts:

1. The level of difficulty of the problem (Easy, Medium, Hard)
2. A numerical difficulty score

The forecasts will be solely on the basis of the text input to the system related to the particular problem description. No tags or submission statistics will be used.

The trained models have been applied in a Streamlit-based web app for the purposes of testing their usability, enabling the user to paste the description of the problem in order to get the prediction instantly.

• Problem Statement

The aim of this project is to develop an intelligent system able to automatically judge the degree of difficulty for a programming tasks text content.

Formally, this is because there are two supervised learning problems:

- Classification Task: Classifying problem difficulty class (Easy, Medium, and Hard).
- Regression Task: Predict a numerical difficulty score

The system should:

- Use only textual features
- Evaluated using standard machine learning evaluation metrics
- Provide predictions using a basic web interface

• Dataset Description

The dataset used in this project is provided in JSONL (JSON Lines) format and contains programming problems annotated with difficulty labels.

A case contains the following fields:

title: Title of the programming problem

description: Problem Statement Text

input_description: Description of the input required by the problem

output_description: Description of the output required by the problem

problem_class: Difficulty class (Easy / Medium / Hard)

problem_score: Numerical difficulty score



The dataset is accessed directly from the following source:

<https://github.com/AREEG94FAHAD/TaskComplexityEval-24>

Due to size considerations, the dataset is not stored locally in the repository and is loaded directly during preprocessing.

- **Data Preprocessing**

Before training the models, several preprocessing steps were applied:

Handling Missing Values

Any missing textual fields were replaced with empty strings to prevent errors during text processing.

Text Combination

To ensure that all relevant information is captured, the following fields were concatenated into a single input feature:

- Problem description
- Input description
- Output description

This combined text serves as the sole input to the machine learning models.

Text Normalization

The combined text is processed using TF-IDF vectorization with English stop-word removal to reduce noise and dimensionality.

- **Feature Engineering**

TF-IDF Vectorization

The **Term Frequency–Inverse Document Frequency (TF-IDF)** technique was used to convert raw text into numerical feature vectors.

Key configuration:

- Maximum number of features: 5000
- Stop words: English

TF-IDF captures both the importance of terms within a problem and their rarity across the dataset, making it suitable for text-based difficulty estimation.

- **Machine Learning Models**

Two independent models were trained to handle classification and regression tasks.

Difficulty Classification Model

- Model Used: Logistic Regression

- Input: TF-IDF feature vectors
- Output: Difficulty class (Easy / Medium / Hard)

Logistic Regression was chosen due to its simplicity, interpretability, and effectiveness for text classification tasks.

Difficulty Score Regression Model

- Model Used: Linear Regression
- Input: TF-IDF feature vectors
- Output: Numerical difficulty score

Linear Regression provides a baseline approach to estimating difficulty scores from textual features.

- **Experimental Setup**

Dataset split:

-> 80% training

-> 20% testing

Random state fixed for reproducibility

Models trained using scikit-learn which helped to:

-> Convert text into numbers (TF-IDF)

-> Train classification model

-> Train regression model

-> Evaluate performance

Evaluation performed on unseen test data

Both models were trained independently using the same TF-IDF feature representation.

- **Evaluation Metrics and Results**

Classification Evaluation

The classification model was evaluated using:

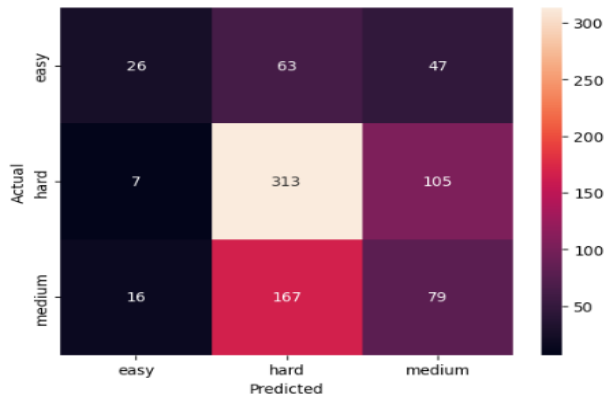
- Accuracy: Measures the proportion of correctly classified problems.
- Confusion Matrix: Visualizes class-wise prediction performance.

```
[17]: accuracy_score(y_class_test, y_pred_class)

[17]: 0.5078979343863913

[18]: cm = confusion_matrix(y_class_test, y_pred_class)

sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=clf.classes_,
            yticklabels=clf.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



The results indicate that the model is able to reasonably distinguish between Easy, Medium, and Hard problems based solely on textual information.

Regression Evaluation

The regression model was evaluated using:

- **Mean Absolute Error (MAE)**
- **Root Mean Squared Error (RMSE)**

These metrics quantify the difference between predicted and actual difficulty scores, with RMSE penalizing larger errors more heavily.

```
[22]: mae = mean_absolute_error(y_score_test, y_pred_score)

mse = mean_squared_error(y_score_test, y_pred_score)
rmse = np.sqrt(mse)

mae, rmse
```

```
[22]: (3.69918641315844, np.float64(4.706225645532768))
```

• Web Interface

A user-friendly web interface was developed using Streamlit.

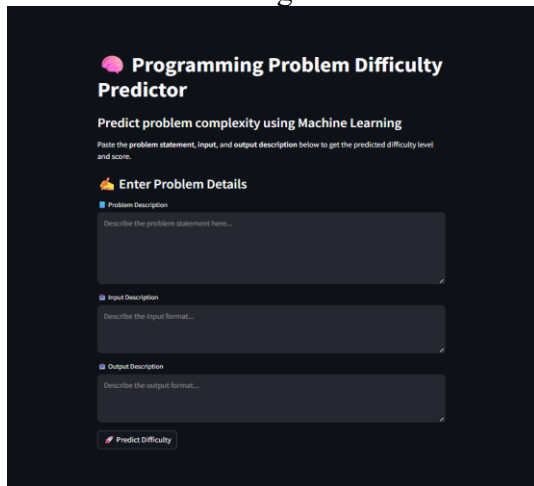
Interface Description The web application allows users to:

1. Enter a problem description
2. Enter input and output formats

3. Click a “Predict Difficulty” button
4. View predicted difficulty class and score

The interface supports both light and dark mode and includes visual enhancements such as:

- Difficulty badges
- Progress indicators
- User feedback messages



The screenshot shows the 'Programming Problem Difficulty Predictor' interface. It has a dark background with white text. At the top, there's a title 'Programming Problem Difficulty Predictor' with a pink brain icon. Below it, a subtitle reads 'Predict problem complexity using Machine Learning'. A small instruction says 'Paste the problem statement, input, and output description below to get the predicted difficulty level and score.' There's a section titled 'Enter Problem Details' with three input fields: 'Problem Description', 'Input Description', and 'Output Description'. Each field has a placeholder text 'Describe the problem statement here...' or similar. At the bottom of this section is a 'Predict Difficulty' button.

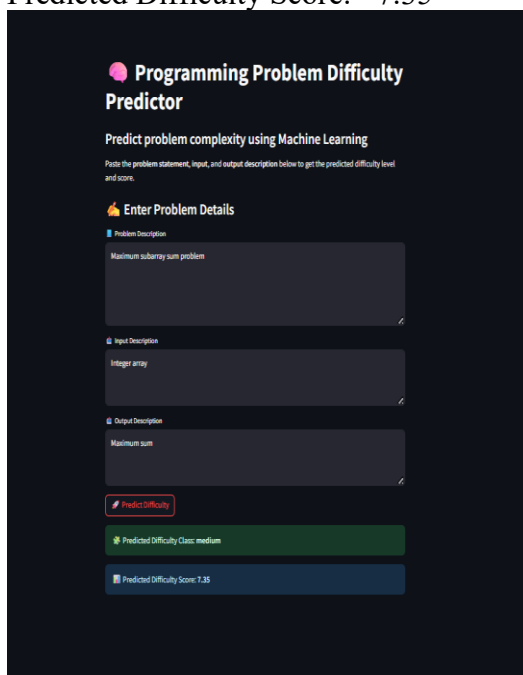
Sample Prediction

Input:

- Problem description: Maximum subarray sum problem
- Input: Integer array
- Output: Maximum sum

Output:

- Predicted Difficulty Class: Medium
- Predicted Difficulty Score: ~7.35



This screenshot shows the same interface as the previous one, but with sample data entered. The 'Problem Description' field contains 'Maximum subarray sum problem', the 'Input Description' field contains 'Integer array', and the 'Output Description' field contains 'Maximum sum'. The 'Predict Difficulty' button is now highlighted with a red border. Below the input fields, there are two output boxes: 'Predicted Difficulty Class: medium' with a green background and a star icon, and 'Predicted Difficulty Score: 7.35' with a blue background and a bar chart icon.



This demonstrates the system's ability to generalize to unseen problems.

• **Discussion**

Since classification and regression models are trained independently, occasional inconsistencies may occur between predicted class and score. This behavior reflects real-world annotation noise and differing optimization objectives and is commonly observed in multi-task ML systems.

Although it is a very simple approach, it works fairly well, especially when it is taken into consideration that it relies on text input only and refrains from using any domain-specific heuristics

• **Conclusion**

This work proves that the difficulty of programming problems can be estimated effectively via machine learning methods based solely on their textual description.

Chief accomplishments include:

- > Excellent text-based difficulty prediction
- > Predicting the numerical difficulty score
- > Launch via the Web Interface
- > Effective evaluation based on objective criteria

The project demonstrates the ability of automatic difficulty measurement and serves as a basis for implementing more sophisticated methods.

• **Future possible improvements**

Possible improvements include:

- Using advanced models such as Random Forest or Gradient Boosting
- Incorporating keyword-based features (e.g., graph, DP, recursion)
- Improving regression accuracy
- Deploying the application online
- Extending the system to multilingual problem descriptions
- Improving the user interface